Simon Sutoris 7542170

1 Typing

1.

$$\dfrac{\dfrac{}{\texttt{Fl}(1) : \texttt{Fl}} \quad \dfrac{\lightning}{\texttt{pow}(\texttt{Fl}(5), \texttt{Fl}(0.5)) : \texttt{Fl}}}{\texttt{Fl}(1) + \texttt{pow}(\texttt{Fl}(5), \texttt{Fl}(0.5)) : \texttt{Fl}} \quad \dfrac{}{\texttt{Fl}(2) : \texttt{Fl}}}{(\texttt{Fl}(1) + \texttt{pow}(\texttt{Fl}(5), \texttt{Fl}(0.5))) \ / \ \texttt{Fl}(2) : \texttt{Fl}}$$

The term is ill-typed, as Ty-Pow requires $t_2 : \texttt{Int}$, but $t_2 = \texttt{Fl}(0.5) : \texttt{Fl}$.

2.

$$\dfrac{\dfrac{}{\texttt{Fl}(3) : \texttt{Fl}} \quad \dfrac{\dfrac{}{\texttt{Int}(4) : \texttt{Int}}}{\texttt{AsFloat}(\texttt{Int}(4)) : \texttt{Fl}}}{\texttt{Fl}(3) * \texttt{AsFloat}(\texttt{Int}(4)) : \texttt{Fl}}}{\texttt{round}(\texttt{Fl}(3) * \texttt{AsFloat}(\texttt{Int}(4))) : \texttt{Int}}$$

3.

$$\dfrac{\dfrac{\dfrac{}{\texttt{Int}(2) : \texttt{Int}}}{\text{textttAsFloat}(\texttt{Int}(2)) : \texttt{Fl}} \quad \dfrac{}{\texttt{Int}(64) : \text{Int}}}{\texttt{pow}(\texttt{AsFloat}(\texttt{Int}(2)), \texttt{Int}(64)) : \texttt{Fl}}$$

4.

$$\dfrac{\lightning}{\texttt{Fl}(4)/\texttt{Int}(2) : \lightning}$$

The term is ill-typed, as Ty-BinOp requires the types $t_1 : \tau$ and $t_2 : \tau$ to be equal, but $t_1 = \texttt{Fl}(4) : \texttt{Fl}$ has a different type than $t_2 = \texttt{Int}(2) : \texttt{Int}$

2 Progress and Preservation

Theorem 1 Preservation

Ty-Int

Term is a value. □

Ty-BinOp, Ty-Float, Ty-Round, Ty-ToFloat, Ty-Pow

Analog to `Ty-Int`.

E-FlOp

$$t_1 = \texttt{Fl}(f_1) \oplus \texttt{Fl}(f_2) : \texttt{Fl} \tag{1}$$
$$t_2 = \texttt{Fl}(f_1 \oplus f_2) : \texttt{Fl} \tag{2}$$

Types of t_1 and t_2 match. □

E-IntOp

$$t_1 = \texttt{Int}(i_1) \oplus \texttt{Int}(i_2) : \texttt{Int} \tag{3}$$
$$t_2 = \texttt{Int}(i_1 \oplus i_2) : \texttt{Int} \tag{4}$$

Types of t_1 and t_2 match. □

E-RoundFl

$$t_1 = \texttt{round}(\texttt{Fl}(f)) : \texttt{Int} \tag{5}$$
$$t_2 = \texttt{Int}(\lfloor f \rceil) : \texttt{Int} \tag{6}$$

Types of t_1 and t_2 match. □

E-IntFl

$$t_1 = \texttt{AsFloat}(\texttt{Int}(i)) : \texttt{Fl} \tag{7}$$
$$t_2 = \texttt{Fl}(i) : \texttt{Fl} \tag{8}$$

Types of t_1 and t_2 match. □

E-PowFl

$$t_1 = \texttt{pow}(\texttt{Fl}(f), \texttt{Int}(i)) : \texttt{Fl} \tag{9}$$
$$t_2 = \texttt{Fl}(f^i) : \texttt{Fl} \tag{10}$$

Types of t_1 and t_2 match. □

E-PowInt

$$t_1 = \texttt{pow}(\texttt{Int}(i_1), \texttt{Int}(i_2)) : \texttt{Fl} \tag{11}$$
$$t_2 = \texttt{Fl}(i_1^{i_2}) : \texttt{Fl} \tag{12}$$

Types of t_1 and t_2 match. □

E-BinOp1

Due to Ty-BinOp, $\oplus$ has the same type as its operands. Due to E-FlOp, E-IntOp, E-RoundFl, E-IntFL, E-PowFl and E-PowInt, t_1 and t_1' in $t_1 \mapsto t_2$ have the same type τ. Therefore $t_1 \oplus t_2$ and $t_1' \oplus t_2$ have the same type τ. □

E-BinOp2, E-Round, E-AsFloat, E-Pow1, E-Pow2

Analog to E-BinOp1.
■

Theorem 2 Progress

Ty-Int, Ty-BinOp, Ty-Float, Ty-Round, Ty-ToFloat, Ty-Pow

These rules are all values. □

E-FlOp, E-IntOp, E-RoundFl, E-IntFl, E-PowFl, E-PowInt

These rules are all reducible terms. □

E-BinOp1

IA: t_1 is a value, i.e. not reducible: $t_1 = t_1'$. Therefore $t_1 \oplus t_2 = t_1' \oplus t_2$.
IS: t_1 is reducible. □

E-BinOp2, E-Round, E-AsFloat, E-Pow1, E-Pow2

Analog to E-BinOp1.
■

3 Designing a type system

1.

$$
\begin{aligned}
t \;:=\; & \\
& \texttt{Int}(n) \\
\mid\; & \texttt{List}(\texttt{Nil}) \\
\mid\; & \texttt{Cons}(t, t) \\
\mid\; & \texttt{tail}(t, t) \\
\mid\; & \texttt{succ}(t, t) \\
\mid\; & \texttt{pred}(t, t)
\end{aligned}
$$

$$\frac{}{\texttt{Int}(\text{n}) : \texttt{Int}}\ \textsc{Ty-Int}$$

$$\frac{}{\texttt{List}(\texttt{Nil}) : \texttt{List}}\ \textsc{Ty-Nil}$$

$$\frac{t_1 : \texttt{Int} \qquad t_2 : \texttt{List}}{\texttt{Cons}(t_1, t_2) : \texttt{List}}\ \textsc{Ty-Cons}$$

$$\frac{\text{t} : \texttt{List}}{\texttt{tail}(\text{t}) : \texttt{List}}\ \textsc{Ty-Tail}$$

$$\frac{\text{t} : \texttt{Int}}{\texttt{succ}(\text{t}) : \texttt{Int}}\ \textsc{Ty-Succ}$$

$$\frac{\text{t} : \texttt{Int}}{\texttt{pred}(\text{t}) : \texttt{Int}}\ \textsc{Ty-Pred}$$

2.

$$
\begin{aligned}
\texttt{tail}(\texttt{Cons}(\texttt{Int}(n), \texttt{List}(vl))) &\mapsto \texttt{List}(vl) && \textsc{Ty-TailCons} \\
\texttt{tail}(\texttt{List}(\texttt{Nil})) &\mapsto \texttt{List}(\texttt{Nil}) && \textsc{Ty-TailNil} \\
\texttt{succ}(\texttt{Int}(n)) &\mapsto \texttt{Int}(n+1) && \textsc{Ty-SuccOp} \\
\texttt{succ}(\texttt{Pred}(n)) &\mapsto \texttt{Int}(n-1) && \textsc{Ty-PredOp}
\end{aligned}
$$

3. Preservation

Ty-Int

Term is a value. □

Ty-Nil, Ty-Cond, Ty-Tail, Ty-Succ, Ty-Pred

Analog to `Ty-Int`.

Ty-TailCons

$$t_1 = \texttt{Cons}(\texttt{Int}(n), \texttt{List}(vl))) : \texttt{List} \tag{13}$$
$$t_2 = \texttt{List}(vl) : \texttt{List} \tag{14}$$

Types of t_1 and t_2 match. □

Ty-TailNil

$$t_1 = \texttt{tail}(\texttt{List}(\texttt{Nil})) : \texttt{List} \tag{15}$$
$$t_2 = \texttt{List}(\texttt{Nil}) : \texttt{List} \tag{16}$$

Types of t_1 and t_2 match. □

Ty-SuccOp

$$t_1 = \texttt{succ}(\texttt{Int}(n)) : \texttt{Int} \tag{17}$$
$$t_2 = \texttt{Int}(n+1) : \texttt{List} \tag{18}$$

Types of t_1 and t_2 match. □

Ty-PredOp

$$t_1 = \texttt{pred}(\texttt{Int}(n)) : \texttt{Int} \tag{19}$$
$$t_2 = \texttt{Int}(n-1) : \texttt{List} \tag{20}$$

Types of t_1 and t_2 match. □

E-Cons1

Due to Ty-TailCons, Ty-TailNil, Ty-SuccOp and Ty-PredOp, t_1 and t_1' in $t_1 \mapsto t_2$ have the same type τ. Therefore $\texttt{Cons}(t_1, t_2)$ and $\texttt{Cons}(t_1', t_2)$ have the same type τ. □

E-Cons2, E-Tail, E-Succ, E-Pred

Analog to E-Cons1.
■

3. Progress

Ty-Int, Ty-Nil, Ty-Cons, Ty-Tail, Ty-Succ, Ty-Pred

These rules are all values. □

Ty-TailCons, Ty-TailNil, Ty-SuccOp, Ty-PredOp

These rules are all reducible terms. □

E-Cons1

IA: t_1 is a value, i.e. not reducible: $t_1 = t_1'$. Therefore $\texttt{Cons}(t_1, t_2) = \texttt{Cons}(t_1', t_2)$.
IS: t_1 is reducible. □

E-Cons2, E-Tail, E-Succ, E-Pred, E-Pow2

Analog to E-Cons1.
■

4 Types and Programming Languages

No questions.

5 Nameless Representations

1. $\lambda.\lambda.1\,0$
2. $\lambda.2\,(\lambda.\lambda.1)$
3. $\lambda.((\lambda.2)\,(\lambda.0))$